

PromptBench

Practice AI-Collaboration Skills Through Graded Business Case Studies

UCS 318 Software Engineering Project Report – Semester Evaluation

Submitted by:

Ridham Goyal (1024030417)

Niloy Sharma (1024030422)

Submitted to:



**THAPAR INSTITUTE OF ENGINEERING &
TECHNOLOGY**
(Deemed to be University)

**Computer Science and Engineering Department
TIET, Patiala**

Acknowledgement

We take this opportunity to express our sincere gratitude to everyone who contributed to the successful completion of our project “PromptBench.”

First and foremost, we would like to extend our heartfelt thanks to our project supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the duration of our project. This encouragement and expertise played a pivotal role in helping us understand the technical challenges and refine our approach at every stage.

We are also grateful to the Thapar Institute of Engineering and Technology (TIET) for providing us with the necessary resources, infrastructure, and learning environment required to develop this project effectively. Their support enabled us to explore, design, and build a fully functioning full-stack system with confidence. Our sincere thanks also go to our peers and classmates for their insightful discussions, suggestions, and motivation during the development of this work. Their input helped us improve several aspects of the project, both technically and conceptually. Lastly, we would like to express our deep appreciation to our families for their constant encouragement, patience, and belief in us. Their support has been a source of inspiration throughout the journey.

We wholeheartedly dedicate this project to everyone who supported us, guided us, and stood by us in completing PromptBench successfully.

S.No.	Student Name	Roll Number
1	Ridham Goyal	1024030417
2	Niloy Sharma	1024030422

Contents

Acknowledgement	1
1 Project Selection Phase	3
1.1 Software Bid	3
1.2 Project Overview	3
2 Analysis Phase	5
2.1 Use Case	5
2.1.1 Use-Case Diagram	5
2.1.2 Use-Case Templates	6
2.2 Activity Diagram and Swimlane Diagram	8
2.3 Data Flow Diagrams (DFDs)	9
2.3.1 DFD Level 0	9
2.3.2 DFD Level 1	9
2.3.3 DFD Level 2	10
2.4 Software Requirement Specification (IEEE Format)	10
2.4.1 Introduction	10
2.4.2 Overall Description	11
2.4.3 Specific Requirements	12
2.5 User Stories and User Cards	14
2.6 Class Diagram	16
2.7 Entity-Relationship (ER) Diagram	17

1. Project Selection Phase

In this phase, the final project idea is chosen based on feasibility, requirements, and expected outcomes. This provides a clear foundation for preparing the software bid.

1.1 Software Bid

Programming Language / Environment Experience

The languages and frameworks the team is most comfortable developing in, used across the project, in order of preference:

1. Python (FastAPI, SQLAlchemy, Pydantic)
2. JavaScript / JSX (React, Vite)
3. SQL (PostgreSQL)

Choices of Projects

	Project Name	Unique Selling Point
First Choice	PromptBench	A benchmarking platform that evaluates how well a candidate <i>collaborates</i> with an AI assistant to solve open-ended business case studies, rather than testing raw coding ability. A strict Helper-AI/Judge-AI isolation boundary ensures the grading rubric and ground truth are never exposed to the candidate-facing assistant, keeping evaluations fair and tamper-resistant.

Additional Remarks / Inputs

The development of PromptBench provided valuable insights into real-world full-stack software development, LLM-integration patterns, and secure system design. Challenges were faced while designing the Helper/Judge isolation boundary, structuring the anti-gaming heuristics, and enforcing a strict JSON schema on LLM-generated evaluation output. In the future, the system can be enhanced with additional case categories, richer analytics dashboards, and organisation-level reporting for recruiters. Overall, this project serves as a strong foundation for a skills-based, AI-native assessment platform.

1.2 Project Overview

Purpose

The main goal of PromptBench is to:

- Give candidates a realistic environment to practice using an AI assistant to solve open-ended business problems.
- Evaluate collaboration skills – clarifying questions, iterative refinement, and hallucination-catching – rather than just the final answer.

- Produce transparent, rubric-grounded feedback that candidates can learn from.
- Protect the integrity of the evaluation by isolating grading logic from the candidate-facing assistant.

Target Users

- **Candidates** – practice solving business case studies with an AI assistant and receive a graded evaluation.
- **Helper AI** – a conversational assistant available to the candidate during the exercise.
- **Judge AI** – a server-side evaluator invoked only at final submission time.

Key Features

- Secure signup / login (bcrypt password hashing + JWT sessions)
- Browsable library of business case studies (data diagnosis, strategy generation, customer reasoning categories, at easy/medium/hard difficulty)
- Interactive case workspace with a Helper AI chat assistant
- Final-answer submission with automated rubric-based grading
- Anti-gaming heuristics (near-zero conversation, too-fast submission, pre-written answer detection)
- Score dashboard with 4-dimension breakdown and judge reasoning
- Attempt history so candidates can track improvement over time

System Overview

PromptBench operates as a web-based platform where:

1. Candidates create accounts and log in.
2. Candidates browse the five seeded case studies and start an attempt.
3. Candidates converse with the Helper AI, which only ever sees the public case context.
4. Candidates draft and submit a final written recommendation.
5. The backend runs anti-gaming checks and then invokes the Judge AI, which independently sees the rubric, hidden ground-truth notes, the full transcript, and the final answer.
6. Candidates receive a structured score (0–100), four dimension sub-scores, judge reasoning, and any anti-gaming flags, and can revisit this history at any time.

2. Analysis Phase

2.1 Use Case

Actors

- Candidate
- Helper AI (candidate-facing assistant)
- Judge AI (server-side evaluator)

Candidate Use Cases

1. Sign Up / Log In
2. Browse Case Studies
3. Start Case Attempt
4. Chat with Helper AI
5. Draft & Submit Final Answer
6. View Evaluation Report
7. View Attempt History

Helper AI Use Case

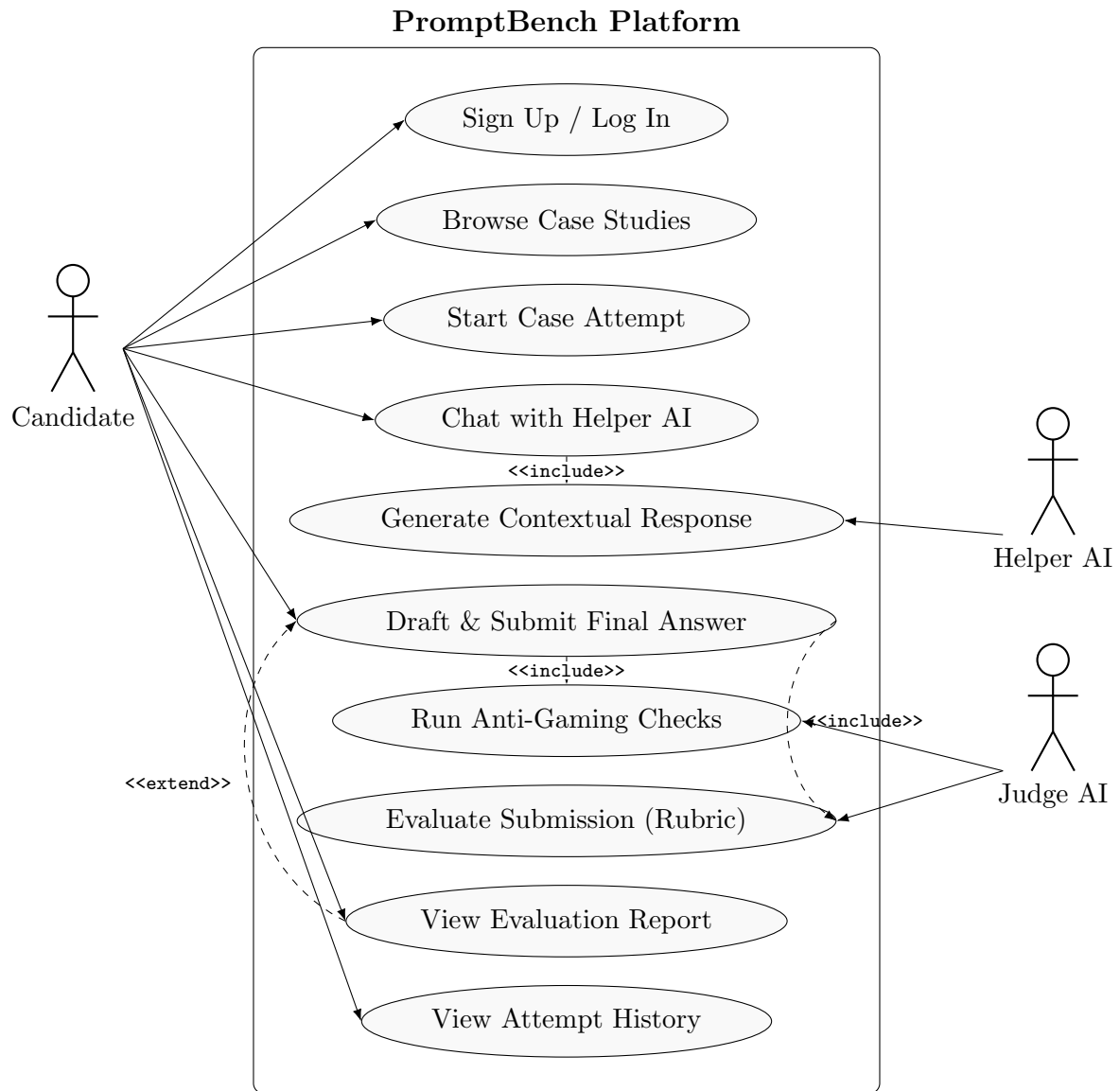
8. Generate Contextual Response

Judge AI Use Cases

9. Run Anti-Gaming Checks
10. Evaluate Transcript & Final Answer Against Rubric

2.1.1 Use-Case Diagram

The Use Case Diagram of PromptBench illustrates the interactions between the Candidate, the Helper AI, and the Judge AI. The candidate signs up/logs in, browses case studies, starts an attempt, chats with the Helper AI, and submits a final answer. Submitting a final answer always *includes* running anti-gaming checks and invoking the Judge AI evaluation, after which the candidate can view the evaluation report and their attempt history. The diagram uses <<include>> and <<extend>> relationships to show mandatory and optional dependencies between use cases.



2.1.2 Use-Case Templates

Use Case: Chat with Helper AI

Actor: Candidate

Description: Candidate exchanges messages with the Helper AI while working on a case.

Preconditions: A case attempt has been started.

Postconditions: A new pair of transcript turns (candidate + assistant) is persisted.

Main Flow:

1. Candidate sends a message.
2. System stores the candidate's turn and loads the full conversation history.
3. System calls the Helper AI with the case context and history.
4. Helper AI's reply is stored and returned to the candidate.

Relationship: Includes → Generate Contextual Response

Use Case: Draft & Submit Final Answer

Actor: Candidate

Description: Candidate submits a final written recommendation for grading.

Preconditions: Submission status is `in_progress` or `evaluation_failed`.

Postconditions: Submission status becomes `submitted` and an Evaluation record is created.

Main Flow:

1. Candidate submits final answer text.
2. System timestamps the submission and loads the full transcript.
3. System runs anti-gaming checks on timing and transcript content.
4. System invokes the Judge AI with rubric, ground truth, transcript, and final answer.
5. System validates the Judge's JSON response against a strict schema and persists the Evaluation.

Relationship: Includes $\rightarrow$ Run Anti-Gaming Checks, Evaluate Submission (Rubric)

Use Case: Run Anti-Gaming Checks

Actor: Judge AI (system)

Description: System flags suspicious submission patterns before/alongside grading.

Preconditions: Final answer has been submitted.

Postconditions: A list of flags (possibly empty) is attached to the evaluation.

Main Flow:

1. Check whether the transcript has fewer than 2 turns (near-zero conversation).
2. Check whether the elapsed time between `started_at` and `submitted_at` is under 30 seconds.
3. Check whether the first candidate message is highly similar (Jaccard ≥ 0.5) to the final answer and over 500 characters (pre-written answer).

Use Case: Evaluate Submission (Rubric)

Actor: Judge AI

Description: Judge AI grades the transcript and final answer against the hidden rubric.

Preconditions: Anti-gaming checks have completed.

Postconditions: An overall score, four dimension scores, and reasoning are produced.

Main Flow:

1. System formats the rubric, ground-truth notes, transcript, and final answer into a prompt.
2. Judge LLM returns a JSON grading report.
3. System parses and validates the JSON against a strict Pydantic schema, retrying extraction if the response is wrapped in extra text.

Use Case: View Evaluation Report / Attempt History

Actor: Candidate

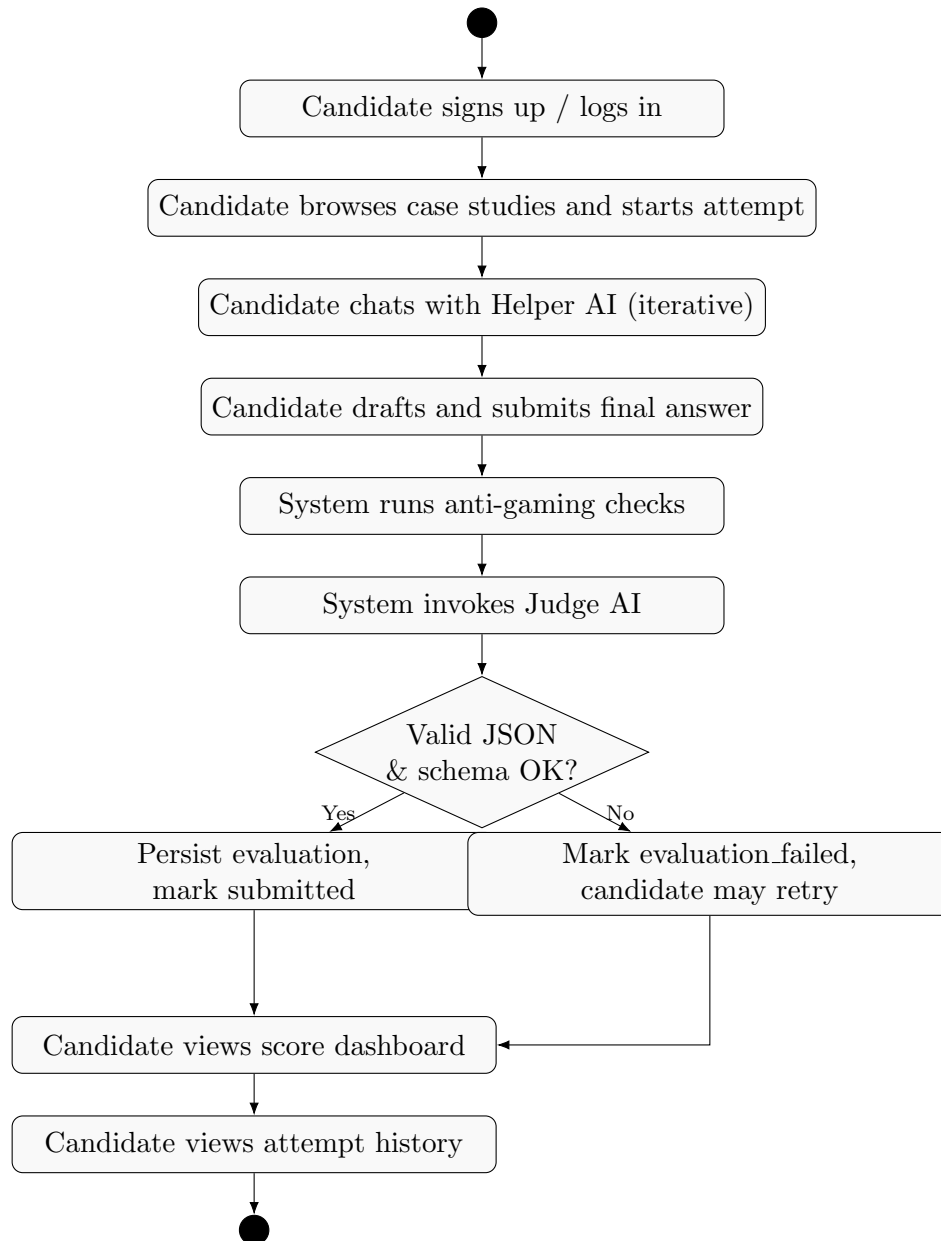
Description: Candidate reviews the score dashboard for a submission, or the list of all past graded attempts.

Preconditions: At least one submission has status `submitted`.

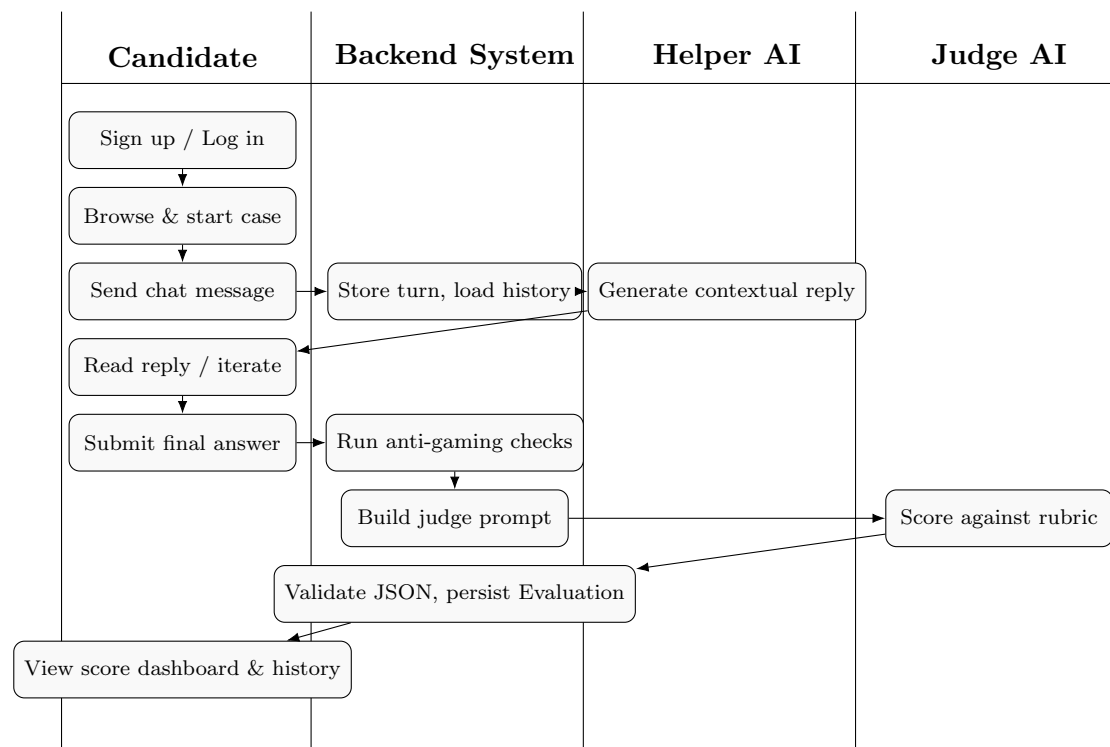
Postconditions: Score, dimension breakdown, judge reasoning, and anti-gaming flags are displayed.

2.2 Activity Diagram and Swimlane Diagram

The Activity Diagram illustrates the complete workflow of PromptBench: a candidate authenticates, starts a case, iterates with the Helper AI, and submits a final answer, which triggers anti-gaming checks and Judge AI grading before results are shown back to the candidate.



This swimlane diagram highlights the same workflow split across the four responsible participants: Candidate, Backend System, Helper AI, and Judge AI.

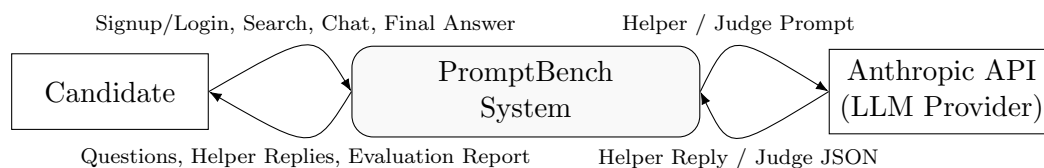


2.3 Data Flow Diagrams (DFDs)

The following figures illustrate the flow of information within PromptBench, from a high-level context down to detailed process-level data movement.

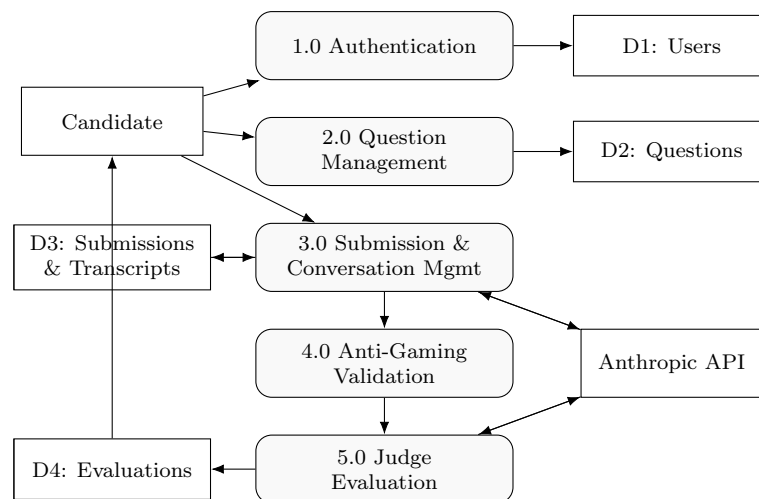
2.3.1 DFD Level 0

The Level 0 DFD provides a context-level overview: the Candidate exchanges case data, chat messages, and final answers with the PromptBench System, and the system in turn calls out to the Anthropic API (LLM provider) to power both the Helper AI and the Judge AI.



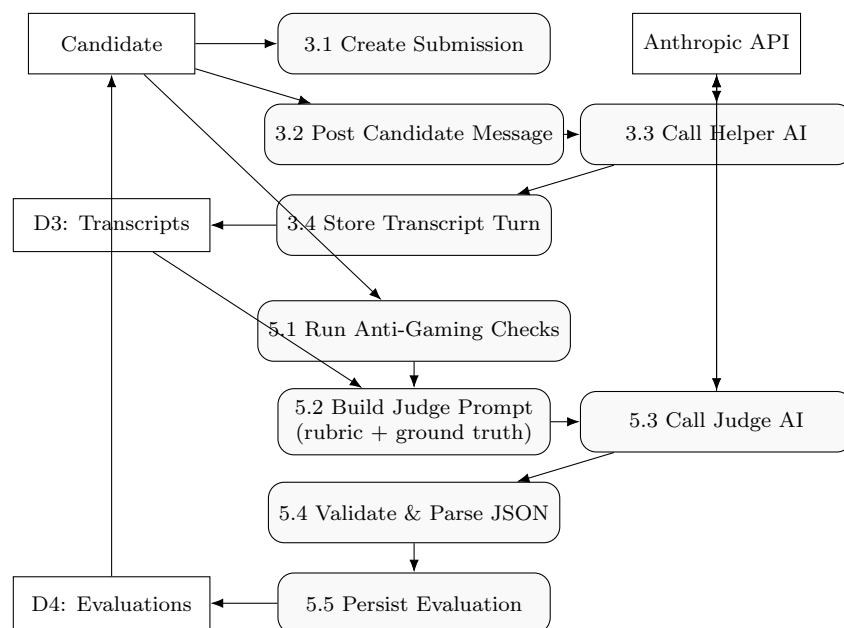
2.3.2 DFD Level 1

The Level 1 DFD breaks the system into its five core processes and four data stores.



2.3.3 DFD Level 2

The Level 2 DFD expands processes 3.0 and 5.0, mirroring the actual backend logic in `submissions.py`, `anti_gaming.py`, and `judge_llm.py`.



2.4 Software Requirement Specification (IEEE Format)

2.4.1 Introduction

Purpose

The purpose of this document is to provide a detailed description of PromptBench, a platform designed to let candidates practice and be graded on how effectively they collaborate with an AI assistant to solve open-ended business case studies. This document serves as a reference for developers (system design and implementation), the Judge AI evaluation logic, and candidates (end users).

Scope

PromptBench is a web-based system that:

- Presents a curated library of business case studies (data diagnosis, strategy generation, customer reasoning) at varying difficulty levels.
- Provides candidates with a Helper AI chat assistant scoped strictly to the public case context.
- Captures the full conversation transcript and final answer for each attempt.
- Grades submissions using a server-side Judge AI against a hidden rubric and ground-truth notes, and screens for gaming behaviour.

Definitions, Acronyms, Abbreviations

- **Case / Question:** A business scenario a candidate is asked to solve.
- **Helper AI:** The candidate-facing conversational assistant.
- **Judge AI:** The server-side evaluator invoked only on final submission.
- **Rubric:** Weighted scoring dimensions used by the Judge AI.
- **Transcript:** The ordered sequence of user/assistant turns for a submission.
- **JWT:** JSON Web Token, used for session authentication.

References

- IEEE SRS Standards
- Software Engineering course material (UCS 318)
- PromptBench project `README.md` and backend source code

Overview

This document describes system features, functional and non-functional requirements, user roles, and system constraints of PromptBench.

2.4.2 Overall Description

Product Perspective

PromptBench is a standalone full-stack web application consisting of:

- Frontend: React 18 + Vite + Tailwind CSS (Login, Signup, Question List, Case Workspace, Score Dashboard, My Attempts pages)
- Backend: FastAPI + SQLAlchemy + Pydantic, exposing REST endpoints under `/api`
- Database: PostgreSQL 16.2
- External Service: Anthropic API (powers both the Helper AI and the Judge AI)

A key architectural principle is **Helper vs. Judge isolation**: the Helper AI never receives rubric schemas, hidden ground-truth documents, grading instructions, or evaluations; the Judge AI is invoked only server-side upon final submission and independently reviews the full transcript, final answer, ground truth, and rubric.

Product Functions

Candidate Functions:

- Sign up / log in
- Browse and start case studies
- Chat with the Helper AI
- Submit a final answer
- View evaluation report and attempt history

System Functions:

- Persist users, questions, submissions, transcript turns, and evaluations
- Run anti-gaming checks (near-zero conversation, fast submission, pre-written answer)
- Call the Judge AI and strictly validate its structured JSON output

User Characteristics

- **Candidate:** Basic technical/business literacy; uses the system to practice case studies.
- **Helper AI / Judge AI:** Automated LLM-backed components; no direct human interaction with the database.

Constraints

- Requires an internet connection and a valid Anthropic API key.
- Grading quality depends on the completeness of the seeded rubric and ground-truth notes.
- Currently limited to a web platform (desktop browser).

Assumptions and Dependencies

- Candidates have basic digital literacy.
- The Anthropic API is reachable and returns well-formed responses.
- PostgreSQL is running and reachable at the configured `DATABASE_URL`.

2.4.3 Specific Requirements

External Interface Requirements

User Interface: Simple, distraction-free case workspace with a chat pane and a final answer draft pane; a score dashboard with dimension breakdowns.

Hardware Interface: Works on any device with a modern web browser.

Software Interface: PostgreSQL database; Anthropic API (`messages.create`); JWT-based authentication.

Functional Requirements

FR1 – Sign Up / Log In Input: email, password, name. Output: JWT access token.

FR2 – List/View Case Studies Input: none / question id. Output: list or detail of case studies.

- FR3 – Start Submission** Input: question id. Output: new submission with status `in_progress`.
- FR4 – Chat with Helper AI** Input: candidate message. Output: stored transcript turns and Helper AI reply.
- FR5 – Submit Final Answer** Input: final answer text. Output: submission timestamped and queued for grading.
- FR6 – Run Anti-Gaming Checks** Input: timestamps, transcript, final answer. Output: list of warning flags.
- FR7 – Evaluate Submission** Input: rubric, ground truth, transcript, final answer. Output: validated overall score, dimension scores, and reasoning.
- FR8 – View Evaluation** Input: submission id. Output: full evaluation report.
- FR9 – View Attempt History** Input: none (current user). Output: list of past graded attempts with scores.

Performance Requirements

- Non-LLM API responses should complete in under 1 second under normal load.
- The system supports multiple concurrent candidates against a shared PostgreSQL instance.
- LLM-bound calls (Helper/Judge) are the dominant latency source and are handled asynchronously from the user's perspective via loading states.

Logical Database Requirements

The system includes the following entities: **User**, **Question**, **Submission**, **Transcript-Turn**, and **Evaluation** (with an embedded **DimensionScores** structure). Each entity has a unique identifier and relationships with other entities, detailed in the Entity-Relationship Diagram (Section 2.7).

Quality Attributes

- **Usability**: Minimal, focused workspace UI.
- **Reliability**: Strict Pydantic schema validation on all Judge AI output; failed evaluations are retryable via the `evaluation_failed` status.
- **Scalability**: Stateless FastAPI backend backed by PostgreSQL.
- **Security**: Bcrypt-hashed passwords, JWT bearer authentication, strict ownership checks on every submission/evaluation route.

Other Requirements

Security: JWT-based authentication (`auth_utils.py`); ownership verification on every submission and evaluation endpoint; rubric and ground-truth notes are never sent to the Helper AI.

Future Enhancements: personalised case recommendations, richer analytics for recruiters/organisations, and a mobile-friendly workspace.

2.5 User Stories and User Cards

User Stories – PromptBench

1. As a candidate, I want to sign up and log in securely so that my progress and attempts are saved to my account.
2. As a candidate, I want to browse the available business case studies so that I can choose one to practice.
3. As a candidate, I want to chat with a Helper AI about the case so that I can ask clarifying questions before proposing a solution.
4. As a candidate, I want to iterate on my approach based on the Helper AI's responses so that I can refine my analysis.
5. As a candidate, I want to submit a final written recommendation so that it can be graded.
6. As a candidate, I want to receive a rubric-based score with reasoning so that I understand my strengths and weaknesses.
7. As a candidate, I want to view my past attempts and scores so that I can track my improvement over time.
8. As the system, I want to run anti-gaming checks on every submission so that low-effort or pre-written answers are flagged.
9. As the Judge AI, I want to grade strictly against a hidden rubric and ground-truth notes so that scores are consistent and fabricated claims are penalised.
10. As a developer, I want the Helper AI and Judge AI to be strictly isolated so that no rubric or grading information ever leaks to the candidate-facing assistant.

Story Cards

Story Card 1

Title: Sign Up / Log In Actor: Candidate

Description: Candidate creates an account or logs in to an existing one.

Acceptance Criteria:

- Duplicate emails are rejected at signup.
- Valid credentials return a usable JWT token; invalid ones return 401.

Story Card 2

Title: Browse Case Studies Actor: Candidate

Description: Candidate views the list of available cases and opens one.

Acceptance Criteria:

- List shows title, category, and difficulty.
- Detail view shows the full prompt and public context.

Story Card 3

Title: Chat with Helper AI Actor: Candidate

Description: Candidate exchanges messages with the Helper AI while solving the case.

Acceptance Criteria:

- Every message and reply is persisted in order.
 - Helper AI only sees the public case context, never the rubric or ground truth.
-

Story Card 4

Title: Submit Final Answer Actor: Candidate

Description: Candidate submits a final recommendation for grading.

Acceptance Criteria:

- An already-submitted submission cannot be re-submitted.
 - Submission triggers anti-gaming checks and Judge AI evaluation.
-

Story Card 5

Title: Run Anti-Gaming Checks Actor: System

Description: System screens submissions for gaming behaviour.

Acceptance Criteria:

- Near-zero conversation, sub-30-second submissions, and pre-written first messages are flagged.
 - Flags are attached to the Evaluation but do not block grading.
-

Story Card 6

Title: Evaluate Submission Actor: Judge AI

Description: Judge AI grades the transcript and final answer against the rubric.

Acceptance Criteria:

- Output is valid JSON matching the strict schema (overall score, 4 dimension scores, reasoning).
 - Malformed output is retried/extracted before failing the submission.
-

Story Card 7

Title: View Evaluation Report Actor: Candidate

Description: Candidate reviews their score, dimension breakdown, and judge reasoning.

Acceptance Criteria:

- Report displays overall score and all four dimension sub-scores.
 - Any anti-gaming flags are visibly surfaced.
-

Story Card 8

Title: View Attempt History Actor: Candidate

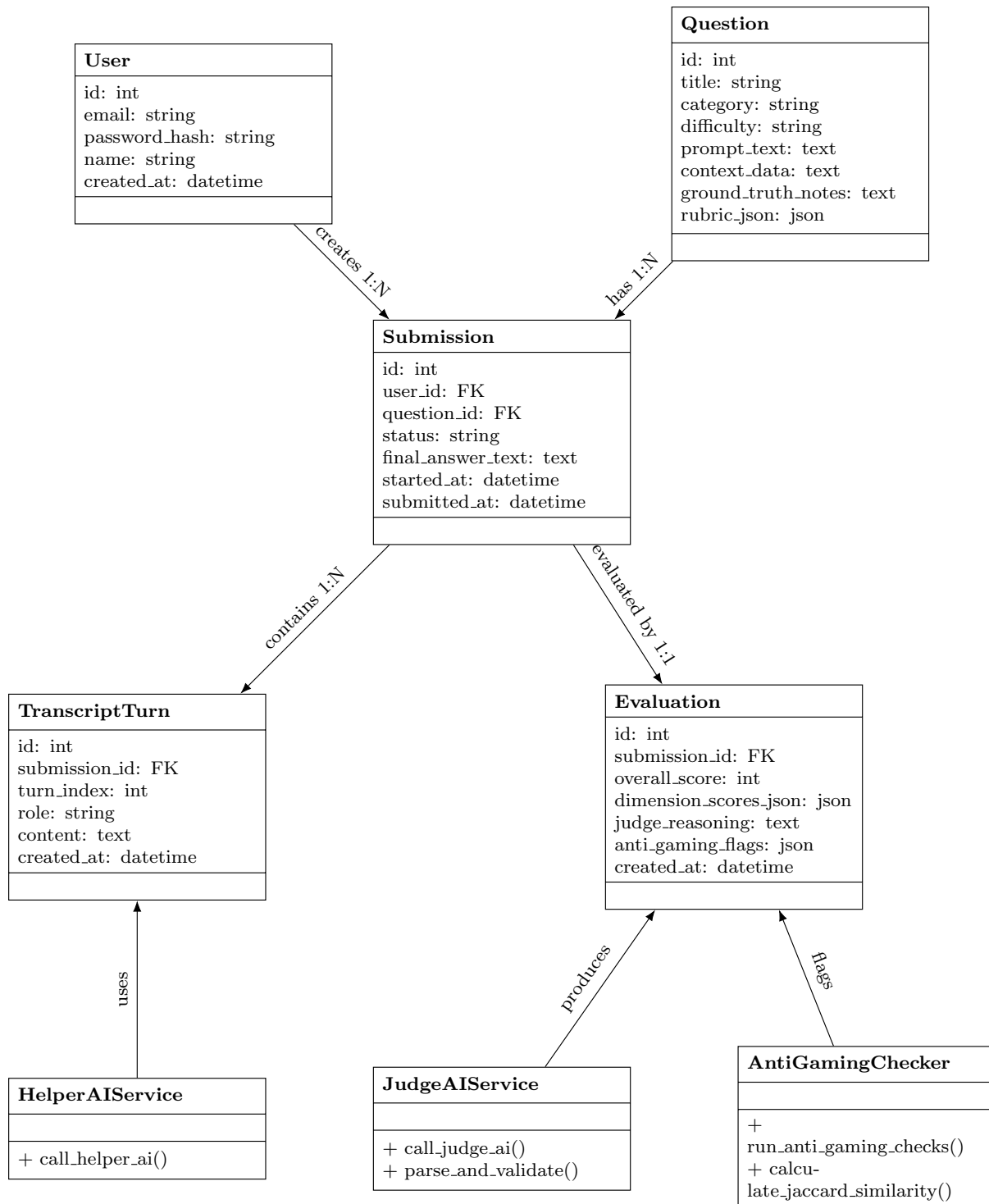
Description: Candidate views all previously graded submissions.

Acceptance Criteria:

- Only the candidate's own submitted (graded) attempts are shown.
- List is ordered by most recent submission first.

2.6 Class Diagram

The Class Diagram represents the static structure of PromptBench: the persistent domain classes (User, Question, Submission, TranscriptTurn, Evaluation) mirror the SQLAlchemy models in `backend/app/models.py`, while `HelperAIService`, `JudgeAIService`, and `AntiGamingChecker` represent the service-layer functions that operate on them.



2.7 Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram below reflects the actual PostgreSQL schema defined in `backend/app/models.py`: a `User` creates many `Submissions`, each `Submission` targets one `Question`, contains many ordered `TranscriptTurns`, and is evaluated by exactly one `Evaluation`, which embeds a `DIMENSION_SCORES` structure (`clarifying_questions`, `iteration_quality`, `hallucination_catching`, `final_answer_quality`).

